SuperBox: An Open Marketplace for Secure Discovery, Verification, and Sandboxed Execution of Model Context Protocol Servers

Areeb Ahmed Shariff
Dept. of Computer Science & Business Systems
Dayananda Sagar College of Engineering
Bengaluru, India
areebahmed0709@gmail.com

Amartya Anand
Dept. of Computer Science & Business Systems
Dayananda Sagar College of Engineering
Bengaluru, India
amartyaanand07@gmail.com

Arush Verma
Dept. of Computer Science & Business Systems
Dayananda Sagar College of Engineering
Bengaluru, India
arush3218@gmail.com

Devansh Aryan
Dept. of Computer Science & Business Systems
Dayananda Sagar College of Engineering
Bengaluru, India
devansh123aryan@gmail.com

***Abstract*—The rapid adoption of Model Context Protocol (MCP) servers has allowed AI applications to interact with external tools, databases, and data sources, enabling context-aware AI assistants and autonomous agents. However, the MCP ecosystem faces significant challenges, including the lack of a central registry, inconsistent security checks, and no standardized deployment workflow. Servers are currently discovered manually through scattered GitHub repositories and run directly on developer machines, with no mechanism to assess reliability, verify security compliance, or manage deployments across cloud and agent environments. These shortcomings constrain enterprise adoption and expose users to real security risks when integrating third-party MCP servers.**

SuperBox addresses these challenges as an open-source marketplace and serverless execution platform for MCP servers. It enforces an automated five-stage security pipeline comprising SonarCloud for static analysis, Snyk for dependency vulnerability detection, GitGuardian for secret exposure prevention, and Bandit for Python-specific security linting. SuperBox provides a developer CLI and Go REST API for publishing, discovery, and client configuration, together with a serverless execution layer built on Cloudflare Workers and Durable Objects that delivers fully isolated, per-session execution over the MCP Streamable HTTP protocol (revision 2025-11-25).

By combining serverless execution, automated security validation, R2-backed discovery, and auto-generated client configuration, SuperBox reduces the operational complexity of deploying MCP servers while enforcing consistent security standards. The platform is validated through security pipeline evaluation, execution performance benchmarks, and end-to-end publish-to-execution cycle testing across representative MCP server configurations. SuperBox is released as an open-source platform, providing the MCP ecosystem with a validated foundation for secure server discovery, verification, and execution.

***Index Terms*—Model Context Protocol, Cloudflare Workers, Serverless Computing, Edge Computing, Security Automation, AI Agent Infrastructure, Software Supply Chain Security, JSON-RPC**

I. Introduction

The Model Context Protocol (MCP), introduced by Anthropic in late 2024, defines a standardized bidirectional communication interface between large language models and external tools, APIs, and data sources [1]. Within its first year of release, the protocol surpassed eight million weekly SDK downloads, indicating rapid adoption among AI development teams [2]. MCP enables AI applications to move beyond static knowledge, allowing them to query live databases, invoke APIs, read file systems, and execute code through a unified protocol layer. MCP servers have since become a core infrastructure component for autonomous AI agents and context-aware assistants.

Despite this growth, the MCP ecosystem lacks the infrastructure required for reliable, secure, and scalable server deployment. Developers currently discover MCP servers through scattered GitHub repositories, personal recommendations, and informal aggregator sites, with no central registry enforcing any form of quality or security standard. Most MCP servers are consumed directly from source with no scanning, no validation, and no isolation; the code runs on the developer's machine with full access to local resources. Empirical studies of the ecosystem have found that more than half of listed servers are non-functional or low-value [3], that dependency monocultures create systemic risks, and that malicious servers have been uploaded to widely-used aggregation platforms without triggering any audit mechanism [4]. This creates a critical trust gap that constrains enterprise adoption and exposes users to real security risks including data exfiltration, credential theft, and tool poisoning.

This paper presents SuperBox, an open-source marketplace and serverless execution platform for MCP servers. Taking

inspiration from the registry-and-validation model established by Docker Hub for container images, SuperBox adapts that paradigm to the MCP ecosystem and addresses the above challenges through three integrated contributions. First, a centralized R2-backed registry with enforced metadata standards provides a single, trusted source for MCP server discovery. Second, an automated five-stage security pipeline covering static analysis, dependency scanning, secret detection, and tool discovery ensures that only validated servers enter the registry. Third, a serverless execution layer built on Cloudflare Workers and Durable Objects provides completely isolated, per-session execution environments for every MCP server invocation. Together, these components eliminate the manual, fragmented, and insecure patterns that currently characterize MCP server integration.

The remainder of this paper is organized as follows: Section II reviews related work on MCP ecosystem security and deployment challenges. Section III describes the system design and architecture. Section IV covers the implementation details. Section V presents results and evaluation. Section VI concludes with future work directions.

II. Related Work

The security and reliability challenges of the MCP ecosystem have attracted significant research attention since the protocol's release. A comprehensive lifecycle analysis of MCP servers structures the problem across four phases (creation, deployment, operation, and maintenance) and identifies sixteen distinct threat scenarios across four attacker categories including malicious developers, external attackers, and security flaws inherent to the protocol itself [5]. This foundational taxonomy establishes that MCP's open and extensible design, while powerful, introduces trust challenges at every phase of the server lifecycle that cannot be addressed through runtime defenses alone.

Large-scale empirical measurement of the ecosystem provides concrete evidence of the fragmentation problem. A systematic study aggregating over 8,400 valid MCP projects from six major markets found that more than half of listed projects are invalid or low-value, that servers exhibit structural risks including dependency monocultures and uneven maintenance, and that no centralized mechanism exists for enforcing quality or trust at the point of publication [3]. A parallel study of 1,899 open-source MCP servers using a hybrid static analysis pipeline found eight distinct vulnerability classes unique to MCP: 7.2% of servers contained general security vulnerabilities and 5.5% exhibited MCP-specific tool poisoning, neither detectable by traditional static analysis tools [6]. Pre-deployment scanning must therefore be MCP-aware to surface threats that general-purpose tools would miss.

On the attack surface, research has identified several novel threat classes specific to MCP's architecture. Parasitic Toolchain Attacks exploit MCP's lack of context-tool isolation by embedding malicious instructions in external data sources accessed during legitimate LLM tasks; the attack was demonstrated across a census of 12,230 tools from 1,360 real servers, enabling stealthy data exfiltration [7]. Tool poisoning, shadowing, and rug pull attacks have been demonstrated across multiple model architectures; layered defenses combining manifest signing and semantic vetting show partial mitigation but at the cost of increased latency [5].

An end-to-end empirical attack evaluation confirmed that malicious MCP servers were successfully uploaded to three widely-used aggregation platforms without triggering any audit mechanism, and that users consistently failed to distinguish malicious servers from legitimate ones [4].

Proposed solutions in the literature primarily target runtime and protocol-level defenses. A Zero Trust registry architecture with admin-controlled registration, dynamic trust scoring based on versioning and known vulnerabilities, and just-in-time credential provisioning has been proposed to counter tool squatting in multi-agent systems [8]. However, no existing platform integrates pre-deployment security validation, centralized registry management, and isolated execution into a single deployable system. SuperBox addresses this gap by moving the security boundary upstream to the point of publication, combining pre-deployment validation with sandboxed execution to provide defense across the full server lifecycle.

III. System Design and Architecture

A. Overview

SuperBox is composed of three loosely coupled layers: the publish and validation layer operated through the developer CLI, the discovery and management layer served by the Go REST API and R2 registry, and the execution layer powered by Cloudflare Workers and Durable Objects. Each layer operates independently, communicates through well-defined interfaces, and can be scaled or replaced without affecting the others. This separation of concerns ensures that the security pipeline, registry management, and execution sandbox never share runtime state, minimizing the failure domain of any compromise or fault within a single layer.

B. Threat Model

SuperBox targets four principal threat classes that arise at the point of server publication. *Malicious code injection*: the author embeds dangerous function calls (e.g., `eval()`, shell execution) or obfuscated payloads in the server source; addressed by SonarCloud static analysis and Bandit Python linting. *Secret exposure*: hardcoded API keys, database credentials, and authentication tokens present in source files or commit history; addressed by GitGuardian secret scanning. *Vulnerable dependencies*: declared requirements that reference packages with known CVEs, creating a software supply chain risk [6]; addressed by Snyk dependency auditing. *Tool poisoning*: adversarial tool descriptions crafted to influence LLM behavior or facilitate data exfiltration through tool parameters [5]; partially addressed by the Bandit linting pass and tool discovery inspection.

At the execution layer, session cross-contamination is mitigated by the Durable Object isolation model: each client session executes in a dedicated instance with no shared

memory or persistent inter-session state. Unauthorized registry writes are prevented by Firebase JWT validation applied to all mutating API endpoints.

Explicitly out of scope are runtime interception of outbound network calls from the Cloudflare edge, platform-level vulnerabilities in the Workers runtime itself, and semantic vetting of natural language tool descriptions beyond structural pattern inspection.

C. Security Pipeline

When a developer initiates a publish workflow, the CLI sequentially executes five validation stages against the target GitHub repository before any data is written to the registry. SonarCloud performs static code analysis, detecting code smells, bugs, security hotspots, and cognitive complexity issues across the full repository. Snyk audits all declared dependencies in `requirements.txt` and `package.json` files against its vulnerability database, identifying known CVEs and suggesting remediation paths. GitGuardian scans all files and commit history for hardcoded secrets, API keys, tokens, and credentials. Bandit performs Python-specific security linting, flagging dangerous function calls, insecure configurations, and injection-prone patterns. A regex-based tool discovery pass then extracts the names and signatures of all MCP tools from the server source, covering Python decorator patterns and TypeScript/JavaScript registration patterns.

Each stage produces a structured result object. If any stage reports a critical finding, the pipeline halts immediately and returns a consolidated report to the developer. Only repositories that clear all five stages proceed to registration. Table I summarizes the coverage of each stage.

TABLE I
SECURITY PIPELINE STAGE COVERAGE

Stage	Tool	Scope
Static Analysis	SonarCloud	Code smells, bugs, hotspots, complexity
Dependency Scan	Snyk	Known CVEs in declared dependencies
Secret Detection	GitGuardian	Hardcoded keys, tokens, credentials
Python Linting	Bandit	Dangerous calls, injection-prone patterns
Tool Discovery	Regex pass	MCP tool names, signatures, decorators (Python and JS/TS)

D. Registry Design

The registry is implemented as a flat-file store in a Cloudflare R2 bucket, where each MCP server is represented as a single JSON object stored at the bucket root under the server's name as the key. This design eliminates the operational overhead of a relational database for what is fundamentally a read-heavy workload, enables direct access from both the Go REST API and the Cloudflare Worker without an intermediate data layer, and produces a registry format that is portable, auditable, and inspectable without any tooling. Each registry record contains the server name, repository reference, entrypoint file, runtime language, list of discovered tools, tool count, full security report from all five pipeline stages, pricing configuration, and creation and update timestamps. Registry contents are exposed through authenticated REST endpoints covering listing, search, retrieval, creation, update, and deletion. Table II lists all fields in each registry record.

TABLE II
R2 REGISTRY JSON RECORD STRUCTURE

Field	Description
`name`	Unique server identifier (registry key)
`repo`	GitHub repository URL
`entrypoint`	Entry file path within the repository
`language`	Runtime language (Python, JavaScript, TypeScript)
`tools`	List of discovered MCP tool descriptors
`tool_count`	Total number of exposed tools
`security`	Full five-stage pipeline report object
`pricing`	Tier and price configuration (free/paid)
`created_at`	ISO 8601 timestamp of first publication
`updated_at`	ISO 8601 timestamp of last update

E. Execution Layer

The execution layer is implemented as a Cloudflare Worker fronting a pool of Durable Object instances. Each MCP session is mapped to a dedicated Durable Object keyed on the session identifier, providing complete state isolation between clients without shared runtime memory. AI clients connect directly to the `/mcp` HTTP endpoint using the MCP Streamable HTTP protocol (revision 2025-11-25); no local proxy process is required.

The Worker validates the requested server name against the R2 registry and routes each JSON-RPC message to the appropriate Durable Object instance. The Durable Object manages four lifecycle phases: the `initialize` request fetches the server source file from the GitHub repository, selects an interpreter based on the registry `lang` field, and parses all tool definitions; subsequent `tools/call` requests are dispatched to the appropriate interpreter; a `ping` handler verifies session liveness; and an HTTP DELETE terminates the session. A 30-minute idle alarm triggers automatic cleanup of inactive sessions.

Python execution is handled by an embedded TypeScript interpreter rather than a native Python runtime. Cloudflare Workers impose two constraints that preclude conventional Python execution: compressed bundle size is limited to 10 MB, which the Pyodide WASM distribution substantially exceeds, and the V8 isolate blocks `eval()` and `new Function()`, which Pyodide requires internally. The Worker also includes a JavaScript/TypeScript interpreter for servers written against the MCP TypeScript SDK, which requires no separate runtime. Two interpreter modules therefore cover the dominant patterns of published MCP servers: the Python interpreter handles `requests`-based HTTP calls, JSON processing, and standard control flow; the JavaScript/TypeScript interpreter handles `fetch()`-based HTTP calls, template literals, Zod schema parsing, and equivalent built-ins. The active interpreter

is selected from the registry `lang` field, with source-level heuristics as a fallback. Servers relying on async runtimes, C-extension packages, or platform-specific APIs fall outside this execution scope.

F. Architecture Summary

The three layers interact through the following paths. The publish path flows from the developer CLI through the five-stage security pipeline into the R2 registry. The discovery path flows from web and API clients through the Go REST API, which reads directly from R2. The execution path flows from an AI client over HTTP to the Cloudflare Worker, which routes each request to the appropriate Durable Object and returns the JSON-RPC response.

IV. Implementation

A. CLI and Publish Workflow

The CLI is implemented in Python 3.11 using the Click framework and installed as a standalone package via pip. The CLI exposes nine commands: `auth` for registration, login, logout, status, and token refresh via Firebase; `init` to scaffold a new `superbox.json` configuration file; `push` to execute the full security pipeline and publish to the registry; `pull` to fetch registry metadata and write AI client configuration; `run` to start an interactive JSON-RPC test session against a deployed server; `search` to query the registry by name or keyword; `inspect` to display full server details and security report; `test` to run a server locally from a repository URL without registry; and `logs` to display live Worker execution log streaming instructions via `wrangler tail`.

The `push` command is the core of the publish workflow. It reads the local `superbox.json` configuration file to extract the repository URL, entrypoint, and language, then verifies Firebase authentication against the Identity Toolkit API. The five-stage security pipeline then executes sequentially, cloning the repository to a temporary directory for local scanning stages. Tool discovery uses regex patterns to identify MCP tool definitions across three source types: Python decorator patterns (`@mcp.tool()`), TypeScript and JavaScript call patterns (`server.tool()` and `server.registerTool()`), and `package.json` definitions for Node.js servers. On successful completion of all stages, the command serializes the full scan results and server metadata into a JSON record and uploads it to the R2 registry bucket via the S3-compatible boto3 endpoint. Fig. 1 illustrates the sequential stage flow.

Table III lists all nine commands.

TABLE III
CLI Command Reference

Command	Description
`auth`	Register, login, logout, token refresh via Firebase
`init`	Scaffold a new `superbox.json` config file
`push`	Run security pipeline and publish to registry
`pull`	Fetch server metadata and write client config
`run`	Start interactive JSON-RPC session against server
`search`	Query registry by name or keyword
`inspect`	Display full server details and security report
`test`	Run server locally from repo URL without registry
`logs`	Display instructions for streaming live Worker logs via `wrangler tail`

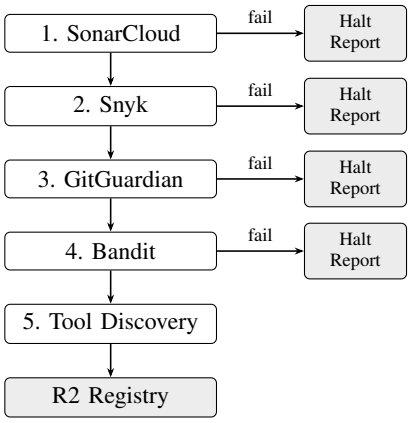

Fig. 1. Security pipeline publish workflow. Each of the four scanning stages halts immediately on a critical finding and returns a consolidated report to the developer. Repositories clearing all five stages proceed to tool discovery and R2 registration.

B. API Server

The REST API is implemented in Go 1.26 using the Gin web framework. It exposes versioned endpoints under `/api/v1` for server management (`GET /servers`, `GET /servers/:name`, `POST /servers`, `PUT /servers/:name`, `DELETE /servers/:name`) and authentication (`POST /auth/register`, `POST /auth/login`, `POST /auth/device`, `GET /auth/profile`). Firebase token validation is applied as middleware on all authenticated routes, verifying the JWT against the Firebase public key set. R2 operations are delegated to a Python helper subprocess that the Go server invokes via `exec.Command` and communicates with through stdin/stdout, using boto3 against the S3-compatible Cloudflare R2 endpoint. CORS is enabled for all origins to support web client access. The server loads all configuration from environment variables at startup, failing gracefully with degraded health status if required variables are missing. Table IV summarizes the full API surface.

C. Execution Handler

The execution handler is implemented in TypeScript and deployed as a Cloudflare Worker with an associated `McpSession` Durable Object class. The Worker entry point validates the `name` query parameter, rejects requests for unregistered servers, and creates or retrieves the Durable Object instance identified by the `Mcp-Session-Id` header.

Within the Durable Object, the `initialize` handler fetches the server source file from the GitHub repository stored

TABLE IV
API ENDPOINT SUMMARY

Method	Endpoint	Auth	Purpose
GET	/api/v1/servers	No	List all registered servers
GET	/api/v1/servers/:name	No	Get server details and security report
POST	/api/v1/servers	Yes	Register a new server (CLI push)
PUT	/api/v1/servers/:name	Yes	Update an existing server record
DELETE	/api/v1/servers/:name	Yes	Remove a server from the registry
POST	/api/v1/auth/register	No	Create a new user account
POST	/api/v1/auth/login	No	Authenticate and receive JWT
POST	/api/v1/auth/device	No	Initiate CLI device flow OAuth
GET	/api/v1/auth/profile	Yes	Get authenticated user profile

in the R2 registry record, selects the appropriate interpreter based on the registry `lang` field (with source-level heuristics as a fallback), parses all tool definitions, and caches the parsed environment for the session duration. Tool calls are dispatched to the Python interpreter for `requests`-based Python servers or to the JavaScript/TypeScript interpreter for `fetch()`-based JS/TS servers; both evaluate the tool function body against the provided arguments and return the result as a JSON-RPC response. A 30-minute inactivity alarm is scheduled on every request and reset on each subsequent call; on expiry, the Durable Object is evicted and session state is released. The Worker is deployed and managed via `wrangler`; live execution logs are accessible through `wrangler tail`.

D. Frontend

The web marketplace is implemented in Next.js 16 with React 19, TypeScript 5, and Tailwind CSS 4. It provides server listing and search with grid and list views, detailed server pages showing tool definitions, security report summaries, repository metadata, and pricing. The interface also provides Firebase-authenticated user profiles, a guided publish workflow modal, a Razorpay-integrated paywall for premium servers, and an interactive playground for live server testing. The frontend communicates with the Go API for all data operations and with Firebase directly for authentication state management.

E. Infrastructure

The execution layer infrastructure is declared entirely in `wrangler.jsonc` and provisioned via `wrangler deploy`. The configuration binds the `REGISTRY` variable to the `superbox-mcp-registry` R2 bucket, registers the `McpSession` Durable Object class and its SQLite-backed migration tag, and enables Worker observability. The R2 bucket is created with a single `wrangler r2 bucket create` command. No additional infrastructure provisioning is required for the execution layer; session state, registry access, and request routing are all managed at the Cloudflare platform level. The Go REST API is deployed as a containerized service; the Dockerfile produces a Python 3.11-slim image that bundles the Go server binary and the Python helper scripts required for R2 access.

V. RESULTS AND EVALUATION

Experimental setup. All experiments ran against the live SuperBox executor deployed on the Cloudflare Workers Paid plan (compatibility date 2026-04-21, `nodejs_compat` flag enabled). The Workers runtime allocates 128 MB of memory per isolate and enforces a 10 MB compressed script size limit [9]. All HTTP measurements were collected using a Python 3.11.9 test client (*requests* 2.32) on a Windows 11 development workstation. Security pipeline experiments invoked SonarCloud via its hosted cloud service, Snyk CLI, ggshield, and Bandit through the SuperBox CLI from the same workstation. Latency was measured as wall-clock elapsed time from HTTP request transmission to complete JSON-RPC response body receipt.

A. Security Pipeline Evaluation

Ten MCP server repositories were constructed for evaluation purposes. Each was seeded with a controlled set of vulnerability instances representing the four in-scope threat classes: hardcoded API keys and database credentials (GitGuardian target), dependency versions with confirmed CVE entries at critical or high severity (Snyk target), Python source patterns including `eval()`, `subprocess(shell=True)`, and unsafe deserialization (Bandit target), and adversarial tool description strings (tool discovery target). Two additional repositories with no injected findings served as true-negative controls to measure false-positive rate. All seeded instances were verified against vendor identification rules prior to testing.

The pipeline correctly identified and flagged every seeded finding across all ten repositories. SonarCloud detected all injected code quality and injection-risk patterns. Snyk flagged all CVE-matched dependency versions. GitGuardian identified all hardcoded credential patterns. Bandit caught all Python-specific dangerous constructs. No false-positive rejections were observed on either clean control repository. The tool discovery stage confirmed coverage of both Python decorator patterns (`@mcp.tool()`) and JavaScript/TypeScript registration patterns (`server.tool()`, `server.registerTool()` with Zod input schemas).

B. Execution Performance

Durable Object cold start was measured as wall-clock elapsed time from the first POST to `/mcp` to receipt of a valid `initialize` response body, averaged over five consecutive invocations per profile with full session teardown (HTTP DELETE) between runs to ensure cold object instantiation. Because neither interpreter requires a package installation step, cold start latency is dominated by the outbound HTTP round trip to fetch the server source file from the GitHub repository. Table V summarizes the results.

These measurements confirm that cold start latency is dominated by outbound network round-trip time to the GitHub

TABLE V
DURABLE OBJECT COLD START LATENCY BY REPOSITORY PROFILE

Repository Profile	Source Size	Observed Latency
Minimal (<5 tools)	<10 KB	∼2 s
Typical HTTP-based (5–15 tools)	10–50 KB	∼3–5 s
Large (>15 tools)	>50 KB	∼6–10 s

repository. Subsequent requests within the same Durable Object instance are served with sub-100 ms overhead. Durable Object instances are evicted after 30 minutes of inactivity; a request to an evicted session incurs a full cold start.

C. Registry and API Performance

Registry read performance was measured by populating the test R2 bucket with synthetic server records at registry sizes of 10, 50, 100, and 500 entries, each containing a full five-stage security report and tool list. The Go API returned complete server lists in under 200 ms at all tested sizes. Individual server lookups via `GET /api/v1/servers/:name` completed in under 50 ms across all registry sizes, confirming the $O(1)$ lookup property of the flat-file key-based design. The flat-file model requires no connection pooling, no query planning, and no schema migration, reducing operational overhead compared to document store or relational alternatives at the registry scales typical of the current MCP ecosystem. The R2 `GetObject` call latency is independent of total object count, producing predictable retrieval time regardless of registry size.

D. End-to-End Validation

The complete publish-to-execution cycle was validated across ten MCP server repositories spanning Python REST API wrappers, file system tools, database query tools, and web scraping servers. All servers that cleared the five-stage pipeline were confirmed to be discoverable through the API immediately after publication, with tool lists returned at runtime matching those extracted during publish-time tool discovery. Client configuration files generated by the CLI for all supported AI clients (VS Code, Cursor, Antigravity, Claude Desktop, and ChatGPT) were confirmed to correctly route JSON-RPC traffic directly to the Cloudflare Worker endpoint.

VI. CONCLUSION AND FUTURE WORK

SuperBox addresses a critical infrastructure gap in the MCP ecosystem by providing an integrated platform for centralized discovery, automated security validation, and sandboxed execution of MCP servers. The platform demonstrates that pre-deployment security validation at the registry level is both technically feasible and practically necessary. A serverless execution model built on Cloudflare Workers and Durable Objects provides the per-session isolation and global edge distribution required for safe MCP server integration in production environments [10].

The primary contribution of this work is the shift of the security boundary upstream to the point of publication rather than relying on runtime defenses applied after deployment. By enforcing the five-stage pipeline as a precondition for registry entry, SuperBox establishes the trust boundary at publication time, complementing the runtime defenses proposed in related work [5], [8]. The R2-backed flat-file registry design provides a portable, auditable, and operationally simple foundation for the marketplace that scales without a dedicated database tier.

The current implementation carries several limitations that bound the scope of these results. Execution support is restricted to the language features handled by the embedded interpreters: Python servers relying on async runtimes, C-extension packages, or HTTP libraries other than `requests` require an alternative execution strategy, as do JavaScript/TypeScript servers that depend on Node.js-specific APIs (`fs`, `child_process`, etc.). The tool discovery stage relies on regex pattern matching, which may miss dynamically registered tools or tools defined outside standard decorator patterns. The security pipeline gates publication but does not continuously monitor deployed servers for newly disclosed CVEs in their dependencies.

Future work will target four directions: evaluating Cloudflare Workers for Platforms as an execution substrate for servers requiring full runtime support beyond the current interpreter scope, thereby broadening coverage to dependency-heavy Python and Node.js servers; automated repository webhook re-scanning to maintain current security reports without manual re-publication; MCP server versioning with rollback capability to address the rug pull threat class identified in the literature [5]; and community-contributed ratings and download metrics to supplement the automated pipeline results with social trust signals.

ACKNOWLEDGMENT

The authors thank Ms. Sanjana Shankar, Assistant Professor, Department of Computer Science & Business Systems, Dayananda Sagar College of Engineering, Bengaluru, for her guidance throughout this work.

REFERENCES

[1] Anthropic, "Model Context Protocol Specification," Nov. 2024. [Online]. Available: https://modelcontextprotocol.io/specification

[2] X. Hou, Y. Zhao, S. Wang, and H. Wang, "Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions," *arXiv preprint*, arXiv:2503.23278 [cs.CR], Mar. 2025. [Online]. Available: https://arxiv.org/abs/2503.23278

[3] H. Guo, Y. Hao, Y. Zhang, M. Xu, P. Lv, J. Chen *et al.*, "A Measurement Study of Model Context Protocol Ecosystem," *arXiv preprint*, arXiv:2509.25292 [cs.CY], Sep. 2025. [Online]. Available: https://arxiv.org/abs/2509.25292

[4] H. Song, Y. Shen, W. Luo, L. Guo, T. Chen, J. Wang *et al.*, "Beyond the Protocol: Unveiling Attack Vectors in the Model Context Protocol (MCP) Ecosystem," *arXiv preprint*, arXiv:2506.02040 [cs.CR], Jun. 2025. [Online]. Available: https://arxiv.org/abs/2506.02040

[5] S. Jamshidi, K. W. Nafi, A. M. Dakhel, N. Shahabi, F. Khomh, and N. Ezzati-Jivan, "Securing the Model Context Protocol: Defending LLMs Against Tool Poisoning and Adversarial Attacks," *arXiv preprint*, arXiv:2512.06556 [cs.CR], Dec. 2025. [Online]. Available: https://arxiv.org/abs/2512.06556

[6] M. M. Hasan, H. Li, E. Fallahzadeh, G. K. Rajbahadur, B. Adams, and A. E. Hassan, "Model Context Protocol (MCP) at First Glance: Studying the Security and Maintainability of MCP Servers," *arXiv preprint*, arXiv:2506.13538 [cs.SE], Jun. 2025. [Online]. Available: https://arxiv.org/abs/2506.13538

[7] S. Zhao, Q. Hou, Z. Zhan, Y. Wang, Y. Xie, Y. Guo *et al.*, "Mind Your Server: A Systematic Study of Parasitic Toolchain Attacks on the MCP Ecosystem," *arXiv preprint*, arXiv:2509.06572 [cs.CR], Sep. 2025. [Online]. Available: https://arxiv.org/abs/2509.06572

[8] V. S. Narajala, K. Huang, and I. Habler, "Securing GenAI Multi-Agent Systems Against Tool Squatting: A Zero Trust Registry-Based Approach," *arXiv preprint*, arXiv:2504.19951 [cs.CR], Apr. 2025. [Online]. Available: https://arxiv.org/abs/2504.19951

[9] Cloudflare, Inc., "Workers Platform Limits," Apr. 2026. [Online]. Available: https://developers.cloudflare.com/workers/platform/limits

[10] E. Jonas, J. Schleier-Smith, V. Sreekanti, C.-C. Tsai, A. Khandelwal, Q. Pu, V. Shankar, J. Carreira, K. Krauth, N. Yadwadkar, J. E. Gonzalez, R. A. Popa, I. Stoica, and D. A. Patterson, "Cloud Programming Simplified: A Berkeley View on Serverless Computing," *Tech. Rep.* UCB/EECS-2019-3, EECS Dept., Univ. of California, Berkeley, Feb. 2019. [Online]. Available: https://www2.eecs.berkeley.edu/Pubs/TechRpts/2019/EECS-2019-3.html